Міністерство освіти та науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра інформатики і програмної інженерії

Звіт

з дисципліни «Основи розробки програмного забезпечення на платформі Java»
Комп'ютерний практикум №3.
Розробка колекцій з використанням Collection Framework. Використання інтерфейсів, лямбда-виразів та посилань на методи при обробці колекцій

Виконав: гр. ІП-з41 Черепов Олександр Павлович

Перевірив: старший викладач Дифучина Олександра Юріївна

Оцінка: _____

Дата оцінки: _____

2026

Комп’ютерний практикум №3

Тема роботи: Розробка колекцій з використанням Collection Framework. Використання інтерфейсів, лямбда-виразів та посилань на методи при обробці колекцій.
Мета роботи: Опанувати використання Java Collection Framework для зберігання, фільтрації та сортування об'єктів за допомогою лямбда-виразів, анонімних класів і посилань на методи.

Завдання до роботи

1. Розробити клас, в якому агрегуються об’єкти класу, розробленого на лабораторній роботі 2, у вигляді колекції інтерфейсного типу Collection. В цьому класі розробити методи, які надають можливість (**30 балів**):
   - відшукувати об’єкт, який відповідає заданим вимогам;
   - створювати колекцію, яка містить тільки унікальні елементи;
   - порівнювати об’єкти за різними критеріями якості;
   - виконувати операцію фільтрації об’єктів за заданою ознакою,
   - виконувати визначення середнього значення кількісної ознаки об’єкта.
2. Розробити метод для сортування об’єктів за вибраним критерієм якості, використовуючи різні способи: 1) з анонімним класом, 2) з лямбда-виразом, 3) з посиланням на методи. Зробити висновки про переваги кожного способу (**20 балів**).
3. Розробити клас, в якому агрегуються об’єкти класу, розробленого на лабораторній роботі 2, у вигляді колекції інтерфейсного типу Map. В цьому класі розробити методи, які надають можливість (**30 балів**):
   - фільтрувати об’єкти колекції за вибраною ознакою,
   - видаляти з колекції об’єкти, які не відповідають заданому критерію,
   - визначати для колекції суму (або іншу операцію reduction) заданої кількісної ознаки.
4. Розробити метод для сортування об’єктів послідовно за кількома вибраними ознаками, використовуючи різні способи: 1) з анонімним класом, 2) з лямбда-виразом, 3) з посиланням на методи. Зробити висновки про переваги кожного способу (**20 балів**).

Номер варіанту: 6.

Реалізація

Структура файлів:

```
programme/
├── auditory/
```

```
│   └── Auditory.java
├── employee/
│   ├── Employee.java
│   └── Laborant.java
├── managers/
│   ├── AuditoriesMap.java
│   └── EmployeesCollection.java
└── Main.java
```

Лістинг програми (лише нові/змінені файли):

programme /managers/EmployeesCollection.java:

```
package managers;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.Comparator;
import java.util.HashSet;
import java.util.List;

import employee.Employee;

public class EmployeesCollection {
    private Collection<Employee> employeesList;

    public EmployeesCollection() {
        employeesList = new ArrayList<>();
    }

    public EmployeesCollection(boolean uniqueOnly) {
        if (uniqueOnly) {
            employeesList = new HashSet<>();
        } else {
            employeesList = new ArrayList<>();
        }
    }

    public void addEmployee(Employee e) {
        boolean wasAdded = employeesList.add(e);

        if (!wasAdded) {
            throw new IllegalArgumentException("This employee is already in
the collection.");
        }
    }

    public Collection<Employee> getEmployeesList() {
```

```
        return employeesList;
    }

    public Employee findEmployeeByName(String name) {
        for (Employee employee : employeesList) {
            if (employee.getName().equalsIgnoreCase(name)) {
                return employee;
            }
        }

        return null;
    }

    public Boolean compareEmployeesSalary(Employee e1, Employee e2) {
        if (!employeesList.contains(e1) || !employeesList.contains(e2)) {
            throw new IllegalArgumentException("Some employees are not in the
collection.");
        }

        return e1.getSalary() > e2.getSalary();
    }

    public double getAverageSalary() {
        double totalSalary = 0;
        int validEmployeesCount = 0;

        for (Employee employee : employeesList) {
            if (employee.getSalary() > 0) {
                totalSalary += employee.getSalary();
                validEmployeesCount++;
            }
        }

        if (validEmployeesCount == 0) return 0;

        return totalSalary / validEmployeesCount;
    }

    public Collection<Employee> filterEmployeesByAuditoriesQty(int min) {
        Collection<Employee> returnCollection = new ArrayList<>();

        for (Employee employee : employeesList) {
            int qty = (employee.getAuditories() != null) ?
employee.getAuditories().length : 0;
            if (qty >= min) {
                returnCollection.add(employee);
            }
        }

        return returnCollection;
    }
```

```
    public List<Employee> sortEmployeesBySalaryAC(int sortMethod) {
        if (sortMethod != 0 && sortMethod != 1) {
            throw new IllegalArgumentException("Invalid sort method argument.
Use 0 for DESC and 1 for ASC.");
        }

        List<Employee> sortedList = new ArrayList<>(employeesList);
        Collections.sort(sortedList, new Comparator<Employee>() {
            @Override
            public int compare(Employee e1, Employee e2) {
                if (e1.getSalary() == e2.getSalary()) {
                    return 0;
                }
                if (sortMethod == 1) {
                    return e1.getSalary() > e2.getSalary() ? 1 : -1;
                } else {
                    return e1.getSalary() > e2.getSalary() ? -1 : 1;
                }
            }
        });

        return sortedList;
    }

    public List<Employee> sortEmployeesBySalaryLambda(int sortMethod) {
        if (sortMethod != 0 && sortMethod != 1) {
            throw new IllegalArgumentException("Invalid sort method argument.
Use 0 for DESC and 1 for ASC.");
        }

        List<Employee> sortedList = new ArrayList<>(employeesList);
        if (sortMethod == 1) {
            sortedList.sort((e1, e2) -> Double.compare(e1.getSalary(),
e2.getSalary()));
        } else {
            sortedList.sort((e1, e2) -> Double.compare(e2.getSalary(),
e1.getSalary()));
        }

        return sortedList;
    }

    public List<Employee> sortEmployeesBySalaryMR(int sortMethod) {
        if (sortMethod != 0 && sortMethod != 1) {
            throw new IllegalArgumentException("Invalid sort method argument.
Use 0 for DESC and 1 for ASC.");
        }

        List<Employee> sortedList = new ArrayList<>(employeesList);
        if (sortMethod == 1) {
```

```
            sortedList.sort(Comparator.comparingDouble(Employee::getSalary));
        } else {
            sortedList.sort(Comparator.comparingDouble(Employee::getSalary).r
eversed());
        }

        return sortedList;
    }
}
```

programme/managers/AuditoriesMap.java:

```
package managers;
import auditory.Auditory;

import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public class AuditoriesMap {
    private Map<Integer, Auditory> auditoriesMap;

    public AuditoriesMap() {
        auditoriesMap = new HashMap<>();
    }

    public void addAuditory(Auditory a) {
        auditoriesMap.put(a.getId(), a);
    }

    public Map<Integer, Auditory> getAuditoriesMap() {
        return auditoriesMap;
    }

    public Map<Integer, Auditory> filterAuditoriesByNumber(int min) {
        Map<Integer, Auditory> returnMap = new HashMap<>();

        for (Map.Entry<Integer, Auditory> entry : auditoriesMap.entrySet()) {
            int entryKey = entry.getKey();
            if (entryKey >= min) returnMap.put(entryKey, entry.getValue());
        }

        return returnMap;
    }

    public void removeAuditoryBySubject(String subject) {
        Map<Integer, Auditory> outputMap = new HashMap<>(auditoriesMap);
```

```
        for (Map.Entry<Integer, Auditory> entry : auditoriesMap.entrySet()) {
            if (entry.getValue().getForSubject().equals(subject)) {
                outputMap.remove(entry.getKey());
                break;
            }
        }

        auditoriesMap = outputMap;
    }

    public int getTotalCapacity() {
        int totalCapacity = 0;

        for (Map.Entry<Integer, Auditory> entry : auditoriesMap.entrySet()) {
            totalCapacity += entry.getValue().getCapacity();
        }

        return totalCapacity;
    }

    public Map<Integer, Auditory> sortAuditoriesByCapacityAC(int sortMethod)
{
        if (sortMethod != 0 && sortMethod != 1) {
            throw new IllegalArgumentException("Invalid sort method argument.
Use 0 for DESC and 1 for ASC.");
        }

        List<Map.Entry<Integer, Auditory>> list = new
ArrayList<>(auditoriesMap.entrySet());

        Collections.sort(list, new Comparator<Map.Entry<Integer, Auditory>>()
{
            @Override
            public int compare(Map.Entry<Integer, Auditory> e1,
Map.Entry<Integer, Auditory> e2) {
                if (e1.getValue().getCapacity() ==
e2.getValue().getCapacity()) {
                    return 0;
                }
                if (sortMethod == 1) {
                    return e1.getValue().getCapacity() >
e2.getValue().getCapacity() ? 1 : -1;
                } else {
                    return e1.getValue().getCapacity() >
e2.getValue().getCapacity() ? -1 : 1;
                }
            }
        });

        Map<Integer, Auditory> sortedMap = new LinkedHashMap<>();
```

```
        for (Map.Entry<Integer, Auditory> entry : list) {
            sortedMap.put(entry.getKey(), entry.getValue());
        }

        return sortedMap;
    }

    public Map<Integer, Auditory> sortAuditoriesByCapacityLambda(int
sortMethod) {
        if (sortMethod != 0 && sortMethod != 1) {
            throw new IllegalArgumentException("Invalid sort method argument.
Use 0 for DESC and 1 for ASC.");
        }

        List<Map.Entry<Integer, Auditory>> list = new
ArrayList<>(auditoriesMap.entrySet());

        if (sortMethod == 1) {
            list.sort((e1, e2) ->
Integer.compare(e1.getValue().getCapacity(), e2.getValue().getCapacity()));
        } else {
            list.sort((e1, e2) ->
Integer.compare(e2.getValue().getCapacity(), e1.getValue().getCapacity()));
        }

        Map<Integer, Auditory> sortedMap = new LinkedHashMap<>();
        for (Map.Entry<Integer, Auditory> entry : list) {
            sortedMap.put(entry.getKey(), entry.getValue());
        }

        return sortedMap;
    }

    public Map<Integer, Auditory> sortAuditoriesByCapacityMR(int sortMethod)
{
        if (sortMethod != 0 && sortMethod != 1) {
            throw new IllegalArgumentException("Invalid sort method argument.
Use 0 for DESC and 1 for ASC.");
        }

        List<Map.Entry<Integer, Auditory>> list = new
ArrayList<>(auditoriesMap.entrySet());

        if (sortMethod == 1) {
            list.sort(Map.Entry.comparingByValue(Comparator.comparingInt(Audi
tory::getCapacity)));
        } else {
            list.sort(Map.Entry.<Integer,
Auditory>comparingByValue(Comparator.comparingInt(Auditory::getCapacity)).rev
ersed());
        }
```

```
        Map<Integer, Auditory> sortedMap = new LinkedHashMap<>();
        for (Map.Entry<Integer, Auditory> entry : list) {
            sortedMap.put(entry.getKey(), entry.getValue());
        }

        return sortedMap;
    }
}
```

programme/Main.java:

```
import java.util.Collection;
import java.util.List;
import java.util.Map;

import auditory.Auditory;
import employee.Employee;
import employee.Laborant;
import managers.AuditoriesMap;
import managers.EmployeesCollection;

public class Main {

    private static void printEmployees(Collection<Employee> employees) {
        int index = 1;
        for (Employee employee : employees) {
            System.out.println(index + ". " + employee.getEmployeeData());
            index++;
        }
    }

    private static void printAuditoriesMap(Map<Integer, Auditory> map) {
        if (map.isEmpty()) {
            System.out.println("[Map is empty]");
            return;
        }
        for (Map.Entry<Integer, Auditory> entry : map.entrySet()) {
            System.out.println("Key ID: " + entry.getKey() +
                " -> " + entry.getValue().getForSubject() +
                " (Capacity: " + entry.getValue().getCapacity() + ")");
        }
    }

    public static void main(String[] args) {
        Auditory a1 = new Auditory(1, "Informatics", 70);
        Auditory a2 = new Auditory(2, "Mathematics", 30);
        Auditory a3 = new Auditory(1, "Physics", 50);

        Employee emp0 = new Employee();
        Employee emp1 = new Employee("Jane Smith", 6000, new Auditory[]{a2});
```

```
        Employee lab1 = new Laborant("Alice Johnson", 7000, new
Auditory[]{a2, a3}, 2);
        Employee lab2 = new Laborant("Bob Brown", 5500, new Auditory[]{a1},
1);
        Employee lab3 = new Laborant("Charlie Davis", 6500, new
Auditory[]{a1, a2}, 2);

        System.out.println("=== TASK 1: EMPLOYEE COLLECTION (UNIQUE ONLY)
===");
        EmployeesCollection collection = new EmployeesCollection(true);

        collection.addEmployee(emp0);
        collection.addEmployee(emp1);
        collection.addEmployee(lab1);
        collection.addEmployee(lab2);
        collection.addEmployee(lab3);

        System.out.println("--- INITIAL EMPLOYEE LIST ---");
        printEmployees(collection.getEmployeesList());

        System.out.println("\nAttempting to add a duplicate:");
        try {
            collection.addEmployee(emp1);
        } catch (IllegalArgumentException e) {
            System.out.println("Error caught: " + e.getMessage());
        }

        System.out.println("\n--- SEARCH AND FILTER METHODS TEST ---");

        Employee found = collection.findEmployeeByName("Alice Johnson");
        System.out.println("Search by name 'Alice Johnson': " +
            (found != null ? "Found - ID " + found.getId() + ", Salary: $" +
found.getSalary() : "Not found"));

        System.out.println("\nFiltering: Employees with 2 or more
auditories:");
        Collection<Employee> filtered =
collection.filterEmployeesByAuditoriesQty(2);
        printEmployees(filtered);

        System.out.println("\n--- STATISTICS TEST ---");
        System.out.println("Salary comparison (Alice vs Bob): " +
            (collection.compareEmployeesSalary(lab1, lab2) ? "Alice earns
more" : "Bob earns more"));

        System.out.println("Average salary in collection: $" +
collection.getAverageSalary());

        System.out.println("\n--- SALARY SORTING TEST (DESC) ---");
        List<Employee> sortedStaff = collection.sortEmployeesBySalaryMR(0);
        printEmployees(sortedStaff);
```

```
                System.out.println("\n\n=== TASK 2: AUDITORIES MAP ===");
        AuditoriesMap auditoriesMapManager = new AuditoriesMap();

        System.out.println("--- ADDING AUDITORIES TO MAP ---");
        auditoriesMapManager.addAuditory(a1);
        auditoriesMapManager.addAuditory(a2);
        auditoriesMapManager.addAuditory(a3);
        printAuditoriesMap(auditoriesMapManager.getAuditoriesMap());

        System.out.println("\n--- TOTAL CAPACITY ---");
        System.out.println("Total capacity of all auditories: " +
auditoriesMapManager.getTotalCapacity() + " students");

        System.out.println("\n--- FILTERING (ID >= 2) ---");
        Map<Integer, Auditory> filteredMap =
auditoriesMapManager.filterAuditoriesByNumber(2);
        printAuditoriesMap(filteredMap);

        System.out.println("\n--- SORTING MAP BY CAPACITY (ASC) ---");
        Map<Integer, Auditory> sortedMap =
auditoriesMapManager.sortAuditoriesByCapacityLambda(1);
        printAuditoriesMap(sortedMap);

        System.out.println("\n--- REMOVAL TEST ---");
        System.out.println("Removing 'Mathematics' auditory...");
        auditoriesMapManager.removeAuditoryBySubject("Mathematics");
        printAuditoriesMap(auditoriesMapManager.getAuditoriesMap());
    }
}
```

Результати тестування програми (Employees Collection):

```
=== TASK 1: EMPLOYEE COLLECTION (UNIQUE ONLY) ===
--- INITIAL EMPLOYEE LIST ---
1. ID: 1, Name: Unknown, Salary: $0.0
2. ID: 3, Name: Alice Johnson, Salary: $7000.0, Role: Laborant, Master Key: 2
3. ID: 2, Name: Jane Smith, Salary: $6000.0
4. ID: 4, Name: Bob Brown, Salary: $5500.0, Role: Laborant, Master Key: 1
5. ID: 5, Name: Charlie Davis, Salary: $6500.0, Role: Laborant, Master Key: 2

Attempting to add a duplicate:
Error caught: This employee is already in the collection.

--- SEARCH AND FILTER METHODS TEST ---
Search by name 'Alice Johnson': Found - ID 3, Salary: $7000.0

Filtering: Employees with 2 or more auditories:
1. ID: 3, Name: Alice Johnson, Salary: $7000.0, Role: Laborant, Master Key: 2
2. ID: 5, Name: Charlie Davis, Salary: $6500.0, Role: Laborant, Master Key: 2

--- STATISTICS TEST ---
Salary comparison (Alice vs Bob): Alice earns more
Average salary in collection: $6250.0

--- SALARY SORTING TEST (DESC) ---
1. ID: 3, Name: Alice Johnson, Salary: $7000.0, Role: Laborant, Master Key: 2
2. ID: 5, Name: Charlie Davis, Salary: $6500.0, Role: Laborant, Master Key: 2
3. ID: 2, Name: Jane Smith, Salary: $6000.0
4. ID: 4, Name: Bob Brown, Salary: $5500.0, Role: Laborant, Master Key: 1
5. ID: 1, Name: Unknown, Salary: $0.0
```

Результати тестування програми (Auditories Map):

```
=== TASK 2: AUDITORIES MAP ===
--- ADDING AUDITORIES TO MAP ---
Key ID: 1 -> Informatics (Capacity: 70)
Key ID: 2 -> Mathematics (Capacity: 30)
Key ID: 3 -> Physics (Capacity: 50)

--- TOTAL CAPACITY ---
Total capacity of all auditories: 150 students

--- FILTERING (ID >= 2) ---
Key ID: 2 -> Mathematics (Capacity: 30)
Key ID: 3 -> Physics (Capacity: 50)

--- SORTING MAP BY CAPACITY (ASC) ---
Key ID: 2 -> Mathematics (Capacity: 30)
Key ID: 3 -> Physics (Capacity: 50)
Key ID: 1 -> Informatics (Capacity: 70)

--- REMOVAL TEST ---
Removing 'Mathematics' auditory...
Key ID: 1 -> Informatics (Capacity: 70)
Key ID: 3 -> Physics (Capacity: 50)
```

Висновок

Під час виконання комп’ютерного практикуму було закріплено навички роботи з Java Collection Framework на прикладі агрегування об'єктів в інтерфейсах Collection та Map. Успішно реалізовано методи додавання, фільтрації, пошуку, видалення та редукції.

Окрему увагу було приділено порівнянню трьох методів сортування структур даних. Анонімні класи виявилися найбільш громіздким і застарілим інструментом, який перевантажує код зайвими конструкціями, особливо при складному розпаковуванні та сортуванні Map. Лямбда-вирази значно оптимізували процес, дозволивши описати логіку порівняння лаконічно в один рядок завдяки функціональному підходу. Проте найкращим та найчистішим підходом виявилися посилання на методи. Використання вбудованих інструментів, таких як Comparator.comparing(...) у комбінації з Map.Entry.comparingByValue(...), робить код декларативним і максимально легким для читання.